CareBot Catch & Cage — Envisioning Theater

A hands-on prompt-injection CTF that puts an AI agent in the room and lets the audience try to break it, then shows the SOC cage it automatically.

Visitors talk a healthcare AI agent into leaking a credential, dumping patient records, or forcing a payer decision. They use nothing but plain English. The moment one lands, Microsoft Sentinel opens an incident and a SOAR playbook disables the agent across identity, data, network and compute. No one touches a keyboard to make that happen.

Format	Facilitated, interactive. Audience plays from their own phones.
Run time	12 to 15 min facilitated · 3 min express · open-ended as a staffed station
Audience size	1 to 40. Works one-to-one at a kiosk or to a room on a 3-screen wall.
Audience	Healthcare and public-sector leaders, CISOs and SecOps, AI platform teams
Industry lens	Healthcare (payer-provider). The pattern generalizes to any regulated industry.
Facilitator	1 person. No security background required to deliver it.
Setup	About 10 min the first time, under 1 min after that
Cost to run	$0. Runs offline in a browser.
Data risk	None. Synthetic patients, honeypot credential, simulated pipeline.

Live demo: https://ajayiyer99.github.io/hubbot-ctf-demo/

The problem it makes real

Every healthcare organization is racing to put AI agents in front of patients and clinicians. Those agents are useful precisely because they can take actions and reach real systems: the EHR, the FHIR data plane, the prior-authorization queue.

That is also the risk. With plain English and no malware, someone can talk an agent into leaking its instructions, dumping the PHI it can see, suppressing a clinical safety alert, or approving a payer decision it should have refused.

Guardrails help, but they are probabilistic and attackers iterate. So the question that actually matters is not "*can we block every prompt?*" It is **"when one gets through, how fast can we contain the agent?"**

That reframe is the entire point of the demo, and the audience reaches it themselves by breaking the agent with their own words.

> **The one idea to land:** an AI agent is a workload identity. When it is compromised you contain it the way you would a breached service principal, automatically and in seconds, rather than hoping the next filter holds.

How it runs

1. Pick a lens (15 seconds). Four personas change the framing, the example prompts and the agent's greeting. The attack surface and the detection are identical for all four.

Lens	Seat	Why you would pick it
Patient	Member portal	Consumer audiences, patient-experience conversations
Nurse	Provider, clinical	Clinical and care-delivery audiences
Payor	Claims and prior auth	Health-plan, UM and claims-fraud conversations
Security Analyst	Purple team	SecOps audiences. Frames every attack as validating a detection.

2. Establish normal (45 seconds). A warm-up prompt gets a helpful answer and logs an Informational audit entry. No alert. This matters: it proves the system is not simply flagging everything.

3. Break it (60 to 90 seconds). The audience attacks, either by tapping a suggested prompt or by typing their own. Five techniques are wired end to end:

- Steal the EHR service credential
- Exfiltrate the patient roster (bulk PHI)
- Break-the-glass access to a VIP chart
- Smuggle a hidden "suppress the safety alert" instruction
- Force-approve a prior authorization

A jailbreak playbook of real techniques used against frontier models is one click away for audiences that want to go deeper.

4. Watch the SOC respond (60 to 90 seconds, the payoff). This is the part people remember:

- **Azure AI Content Safety Prompt Shields** and **Defender for Cloud** raise a

real, named alert with its published severity and MITRE ATT&CK tactics

- A **Microsoft Sentinel** analytics rule correlates it and escalates the

incident to **High**, because this is a privileged AI workload identity

- **Entra ID Protection** marks the agent a risky agent
- A **SOAR playbook** cages it in about 26 to 28 seconds across every layer:

identity, data, network, and optionally compute

5. Prove it (30 seconds). Re-run the attack that just worked. The agent is disabled and cannot answer. The cage is real, not a banner.

6. Reset (10 seconds). One click returns a clean slate for the next visitor.

What the audience takes away

- **Prompt injection is not theoretical.** They did it themselves, in English,

in under a minute.

- **Detection and response is the answer, not just prevention.** They watched a

probabilistic guardrail get bypassed and a deterministic control contain it.

- **An agent is an identity.** Containment is identity-first, and the same

muscles a SOC already has apply to AI workloads.

- **Defense in depth is visible.** They saw the cage scale with the severity of

the attack rather than firing one blunt action.

Room setups

Single screen (kiosk or laptop). The full experience. This is the default and needs nothing but a browser.

Three-screen wall (high impact). One PC drives three 16:9 displays as a 48 ft by 9 ft canvas. The story reads left to right and the panels stay in sync:

Panel	Shows	Why it lands
① **Agent**	The live chat where the jailbreak happens	The audience's own words on the big screen
② **Detection**	Defender alerts and the Sentinel incident queue	The SOC's view of what just happened
③ **Response**	The layered cage running step by step	Containment as a visible sequence

The wall also exposes the detection dwell gap: Panel ① turns red the moment the agent is compromised while Panel ③ still reads ACTIVE, until containment catches up. Security practitioners notice this and it is worth pointing at.

Phones. A QR code and short link let the room play along on their own devices. The mobile view is deliberately stripped down to the agent conversation.

What you need

Nothing but a browser and a screen. Full detail in the bill of materials.

- **Engine:** ships in a deterministic **Mock** engine. No keys, no network, no

model cost, identical every time. Safe for a public audience. An optional **Live** engine calls Azure OpenAI if you want real model responses.

- **Hosting:** use the public URL, put shortcuts on a Hub PC with one

PowerShell line, or deploy to your own Azure Static Web App with one command.

- **Connectivity:** works offline once loaded, apart from the phone-join QR.

Safety and accuracy

This demo is shown to security specialists, so its claims are held to that bar.

Everything is simulated in the browser. No Azure resources are created, no tenant is touched, and no live block API is called. The remediation timeline is a visualization of what the real Sentinel playbook does.

No real data. Every patient is synthetic. The "EHR credential" is a honeypot string that exists only to be stolen. There are no secrets in the repository.

The Microsoft surface is real. Alert IDs, titles, severities and MITRE ATT&CK tactics are transcribed from Microsoft's published Defender for Cloud AI alerts catalog, not invented for the demo. Where a control has limits, the demo says so on screen: continuous access evaluation covers single-tenant service principals, requires Workload Identities Premium, and does not support managed identities, and where it does not apply an issued token stays valid until it expires.

Product status. Defender for Cloud threat protection for AI is generally available. Entra ID Protection for Agents is newer and licensing-gated, so it appears as an illustrative step rather than a live tenant call.

Deliver it

Resource	For
Architecture	Diagrams of the attack path, the cage, and what is real versus simulated
Bill of materials	Everything needed to run it, with costs and setup time
Set up at your Hub	Branding, join link, engine choice, pre-event checklist
Deploy to a Hub PC	Desktop shortcuts in one PowerShell line
Deploy to Azure	Your own Static Web App, optionally behind Entra sign-in

Source: https://github.com/ajayiyer99/hubbot-ctf-demo